RSCTF TECHNICAL PRACTICE PAPER

HOW RSCTF SCORES KING OF THE HILL: THE CROWN-CYCLE MODEL

Dimas Maulana
RSCTF Project · Competition Platform
Crown-cycle implementation · Manuscript version 2.5 · 14 July 2026
Scoring policy: fixed crown-cycle KotH formula

ABSTRACT

In King of the Hill (KotH), every team attacks one shared service for each challenge. RSCTF's fixed scoring formula divides an epoch into shorter crown cycles: scored periods separated by clean resets. This design stops an early team from keeping the same patched challenge for the entire event. At each cycle boundary, RSCTF finalizes the evidence, destroys the old container, and starts one clean replacement from the same configured image. Scoring resumes only after an automated readiness test confirms that the replacement works. A new claim remains provisional until the checker observes the same team's code and a healthy service in consecutive rounds. When at least one challenger remains, the previous cycle's leader or tied leaders normally receive a one-tick block from that hill. The score is $100R(0.25A + 0.55C + 0.20\sqrt{(AC)})$, where A measures confirmed acquisition, C measures sustained control, and R measures service reliability while the team is responsible. Control has the largest direct weight, and reliability scales the whole score. A blocked tick does not count as an opportunity for the affected team. Reset time, readiness time, incomplete code issuance, and platform faults do not count for any team. Finalized epochs determine rank; unfinished epochs appear only as projections. Equal final scores are resolved by control rate, reliability, confirmed acquisitions, and participation ID.

Keywords: *King of the Hill; capture the flag; cybersecurity competition; crown cycle; qualified capture; service reliability; transactional recovery; RSCTF*

Document status: versioned technical-practice report, not a claim of peer review. Appendix C maps implementation claims to repository source files.

START HERE: KOTH IN 60 SECONDS

The big idea

Every team attacks the same running challenge. Teams do not get separate copies. Become king by placing your team's current code on the shared hill and keeping the challenge working.

A **healthy check** means that RSCTF sees your team code and confirms that the challenge still works.

Play in four steps

1. **Attack the hill:** find a way into the shared challenge.
2. **Place your claim:** copy your current team code from the KotH toolkit and write it to `/koth/king`.
3. **Pass two checks:** RSCTF shows **Provisional** when it first sees your valid code. With the default settings, two consecutive healthy checks confirm you as king.
4. **Hold it:** keep your code there and keep the challenge working. Another team can replace your code and take the crown.

How points work

You earn more by doing three things:

- taking the hill;
- holding it longer, which matters most; and
- keeping it working while you control it.

If the challenge breaks while your team controls it, your whole KotH score goes down.

What starts over

A **crown cycle** is a short block of scoring rounds. At its end, RSCTF pauses scoring, saves the result, removes the old challenge, and opens a clean copy. Your patches and access disappear, and old team codes stop working. Teams receive new codes and compete again. The leading team is normally blocked for the first scored round. If teams tie for the lead, RSCTF blocks every tied leader unless that would leave no challenger.

Read the board

Projected can still change. **Settled** is final and determines the official rank.

New player? Continue with Sections 2.1-2.4, the scoreboard guide in Section 3.6, and Section 7. The later sections explain the exact mathematics, recovery rules, and organizer controls.

Key terms used in this handbook

- The **hill** is the one running challenge shared by every team.
- A **team code** is the current secret value that identifies your claim. The technical sections also call it a **token** or **capability**.
- The **marker** is the `/koth/king` file where a team places its code.
- The **checker** is RSCTF's automated test. It reads the marker and checks that the challenge still works.
- A **crown cycle** is the short scored period between clean resets.

- An **epoch** is a longer group of scored rounds that produces one result.
- When at least one challenger remains, a **cooldown** temporarily blocks the previous cycle's strongest controller or tied controllers from this hill.
- **Readiness** is RSCTF's test that a clean replacement is working before scoring resumes.

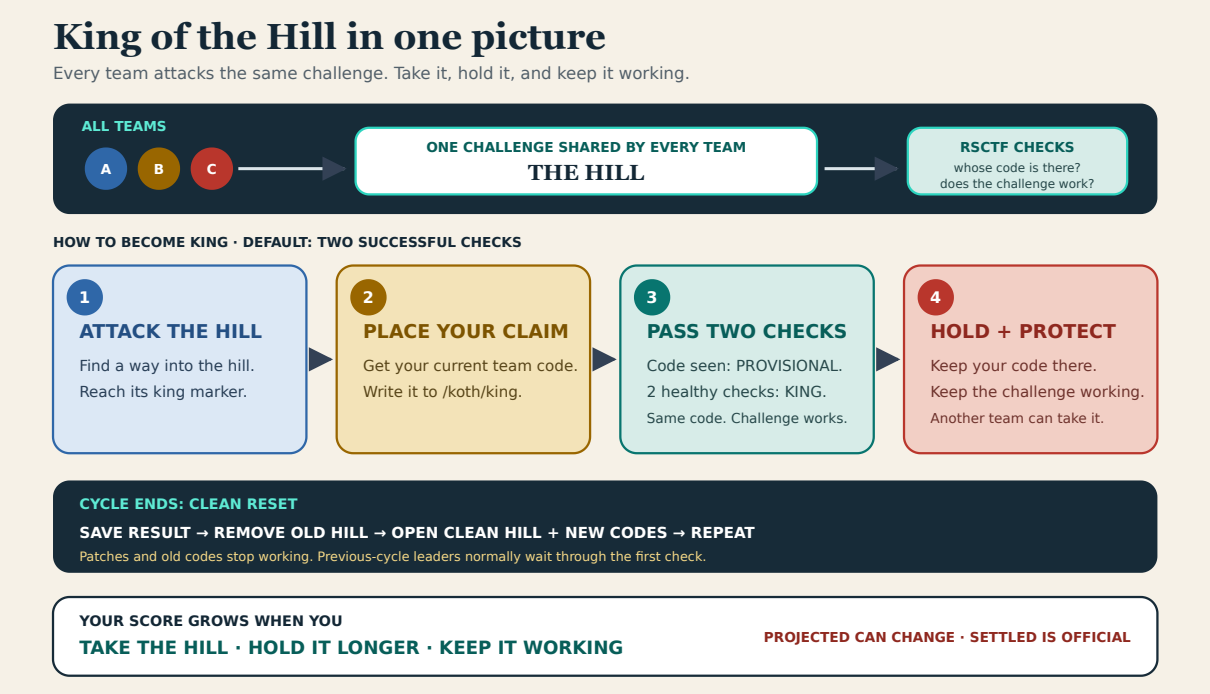

Figure 1. KotH in one picture. All teams attack the same running challenge. Seeing a valid team code creates a provisional claim; two healthy checks confirm the king by default. Teams score for taking the hill, holding it, and keeping it working. A clean reset starts the next cycle.

1. INTRODUCTION

The technical sections use **capability** and **token** for the team code. A **container** is the running shared challenge. These exact names let RSCTF tie each checker result to one team code and one running challenge.

King of the Hill places every team against the same service. A team claims control by writing its exact current capability to the hill's marker file, `/koth/king`. The checker reads that marker and tests the service. The hill is exclusive: one KotH challenge has one active shared container. A takeover changes the holder of that container; it does not create a separate instance for the attacking team.

The fixed formula addresses two fairness problems. An early team could harden the container so thoroughly that the rest of the event becomes a permanent defense exercise. A team could also write its marker briefly and appear to capture the hill even if the service immediately fails or another team takes over at the next check. Fixed resets address the first problem. Consecutive healthy observations, called **qualified capture evidence**, address the second. Patching remains useful, but each patch lasts only until the next clean crown-cycle reset.

The score measures three outcomes during each fixed **epoch**, or scoring block: confirmed acquisition, observed control, and service reliability while a team is responsible. Sustained control receives the largest direct coefficient. Reliability multiplies the complete acquisition-control result, so a team cannot preserve a large offensive score while leaving the service broken.

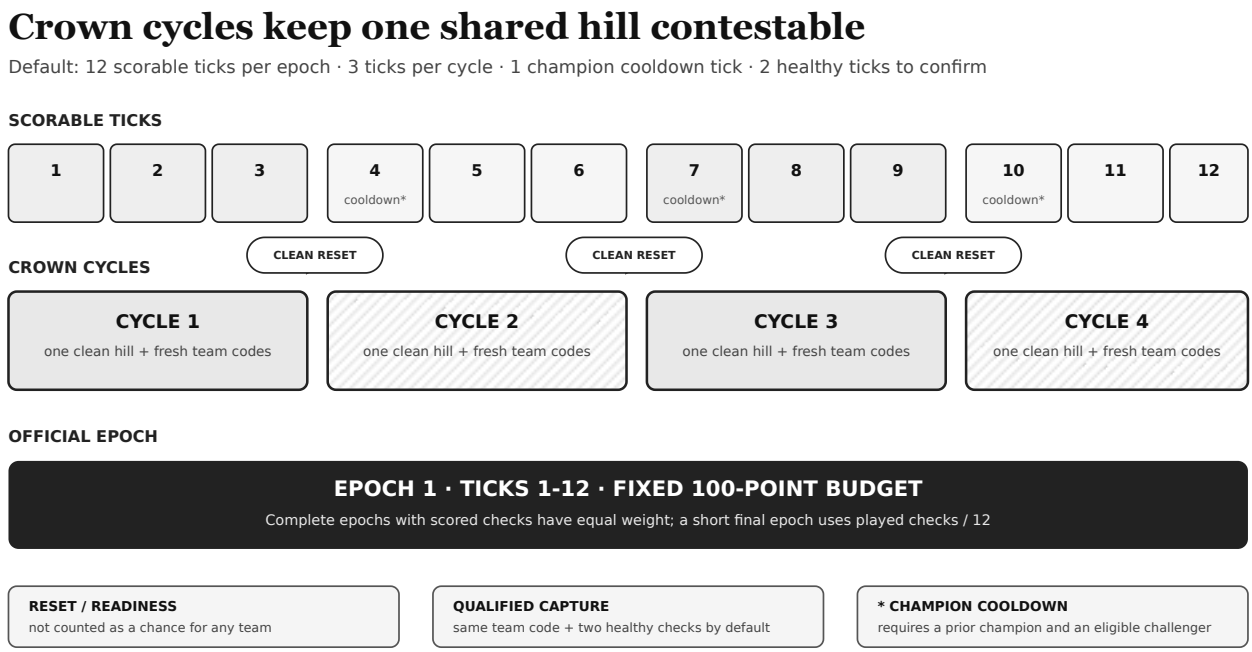

Figure 2. Default RSCTF cadence. A twelve-tick epoch contains four three-tick crown cycles. Each cycle starts only after a pristine replacement passes readiness. RSCTF excludes reset and readiness time from scoring.

1.1 Design requirements

The scoring and reset code must preserve eight requirements:

1. **One hill means one container.** Reset logic never permits two active containers for the same target.
2. **Control must be contestable.** A pristine same-image reset bounds the lifetime of patches, implants, credentials, and filesystem changes.
3. **A capture must be qualified.** By default, two consecutive healthy checker observations distinguish a sustained claim from a momentary marker write.
4. **Sustained control must matter more than capture speed.** Control has a 55% direct coefficient; acquisition has 25%.
5. **Service correctness must constrain the whole score.** Reliability multiplies every acquisition and control contribution.
6. **Forced cooldown must remove opportunity, not assign failure.** A cooled champion's personal denominators omit its blocked tick.
7. **Infrastructure faults must not become player penalties.** Reset, readiness, incomplete token issuance, and `InternalError` evidence are void.
8. **Every transition must be auditable and recoverable.** Each evidence record identifies its game, hill, cycle, reset attempt, container, round, and token window. These identities let operators trace and safely resume interrupted work.

These properties do not prove that every challenge is balanced. Event organizers remain responsible for checker quality, equivalent network access, challenge design, collusion rules, and incident decisions.

1.2 Relation to prior KotH and A&D systems

Bock, Hughey, and Levin (2018) describe an instructional KotH in which teams claim shared machines, become responsible for critical services, and receive points at repeated checks [2]. Their scoring gives later rounds more value because hardened systems should become harder to retake. RSCTF takes a different approach: periodic clean resets preserve opportunities to retake the hill, and every complete epoch containing evidence has equal weight.

CTFd's documented KotH challenge checks an agent at intervals and awards a configured reward to the identifier it reports [3]. RSCTF also reads an identity marker, but it records four parts separately before calculating the score: provisional control, confirmed acquisition, control duration, and service reliability.

FAUST CTF uses Attack & Defense rather than a single shared hill. Its public rules provide an operational comparison because both systems use repeated ticks and service checks [4]. The two formats do not use the same target layout or scoring formula.

2. COMPETITION PROTOCOL

2.1 Shared hill and control marker

RSCTF runs each enabled ranked KotH challenge in one managed container. It publishes the container's unique identity so each checker result identifies the correct running challenge. When official scoring starts, RSCTF records and locks the configured image for scoring history. RSCTF reads the standard claim marker inside that exact container:

```
/koth/king
```

A team follows five steps. Here, the **active reset attempt** identifies the specific clean container created for the current cycle.

1. obtain its current capability for the hill and active reset attempt;
2. exploit or administer the published shared service;
3. write only that capability to `/koth/king`;
4. preserve the behavior required by the functional checker; and
5. keep the marker stable for the configured confirmation streak.

Players do not submit a KotH capture through a separate API endpoint. Writing the marker creates evidence only when the checker reads it from the exact active container before the event deadline.

2.2 Crown-cycle lifecycle

The default values are:

Setting	Default	Valid range
Epoch length	12 ticks	2-64 ticks
Crown-cycle length	3 ticks	At least 1, at most half the epoch, and an exact divisor of the epoch
Previous-champion cooldown	1 tick	0 through cycle length minus 1
Claim confirmation	2 ticks	1 through the cycle length

At each crown-cycle boundary, RSCTF stores every completed phase of one **durable transition** so an interrupted reset can resume without repeating work: **finalize → pause → replace → issue new claim codes → prove readiness → resume.** The phases are:

1. finalize the previous cycle's evidence and select its champion or tied champions;
2. stop assigning checker results and scores to the hill;
3. record an audit receipt and a filesystem diff when the runtime supports it;
4. destroy the old container completely;
5. create one replacement from the same snapshotted challenge image;
6. publish the replacement's exact container identity and network endpoint;
7. clear the previous holder, responsibility, provisional claim, and stale marker state;
8. revoke old claim codes and issue one fresh capability per eligible team and hill;
9. run readiness and the functional checker against the replacement;
10. install the champion cooldown at the VPN/firewall layer when configured; and
11. activate the cycle only after readiness succeeds.

Reset and readiness rounds do not count as scoring opportunities for any team. After a crash, RSCTF resumes from the last stored phase. It does not create a second active container or award the same scoring credit again.

2.3 Qualified capture

The first scorable checker observation of an eligible current capability creates a **provisional claimant**, meaning the checker has seen the team's claim but has not confirmed it. The service verdict can be `Ok`, `Mumble`, or `Offline`. That sample immediately counts as one controlled tick and one responsible tick, but it does not award acquisition credit. Under the default configuration, the checker must then observe the same token, the same team, and an `Ok` service for two consecutive scorable rounds.

A different valid token starts a new streak for its team. `Mumble` or `Offline` resets the current claimant's confirmation progress to zero but leaves the claim provisional. `InternalError`, reset time, readiness time, and incomplete capability issuance create **void evidence**, meaning the sample counts for no team. A void sample pauses confirmation: it neither penalizes the claimant nor erases an earlier healthy step. On confirmation, RSCTF writes one permanent acquisition receipt for that exact cycle and token. Retrying the same operation cannot create a second receipt.

The public board therefore distinguishes three states:

- no observed eligible claim;
- provisional claimant with `progress / required` healthy observations; and
- confirmed king.

2.4 Previous-champion cooldown

RSCTF selects the previous cycle's champion by counting each team's confirmed ticks that were both controlled and healthy. The team with the highest count wins the cycle; the last marker alone does not decide the result. Every team tied for the highest count enters the champion set. RSCTF disables cooldown for that cycle if blocking the full tied set would leave no eligible challenger.

Cooldown starts only after the clean replacement passes readiness. The VPN or firewall blocks each affected team from that hill, and the application rejects its marker as a second check. The block does not affect the rest of the event. By default, cooldown lasts one **authoritative scoring round**: one round counted by the platform scheduler, not a wall-clock timer. RSCTF removes that round from the cooled team's own total of available scoring opportunities. An acquisition window still counts as an opportunity for that team if the window contains a later scorable tick after cooldown.

This rule requires enforceable network isolation. If an external target cannot support a firewall rule for one specific hill, the organizer must reject that configuration or clearly disable

cooldown. The interface must not report an unenforced cooldown as active.

2.5 Tick, cycle, and epoch

Table 1. Operational clocks and scoring effects.

Clock	Definition	Scoring effect
Tick / round	One pass of the scheduler, with at most one permanent checker result per hill.	Records who controlled the hill, who was responsible, and whether the service worked.
Crown cycle	A fixed set of active scoring ticks played on one clean container.	Defines claim codes for that reset, acquisition opportunities, and the cycle champion.
Epoch	A fixed number of scoring ticks combined into one score bounded from 0 to 100.	Produces one equal-weight official score after finalization; a shortened final epoch receives proportional weight.

At the official scoring boundary, the platform stores a fixed copy, or **snapshot**, of the timing settings, accepted teams, enabled hills, service weights, and configured images. Later configuration changes cannot rewrite finalized history.

2.6 Exact attribution and void evidence

A control observation can affect the score only if its game, challenge, round, cycle, reset attempt, token, target, and container identity all match the active hill. An old capability cannot authenticate after a reset. A late checker result from the destroyed container cannot enter the replacement's score.

The checker reads the marker immediately before and after testing the service. If the marker changes during that test, RSCTF does not elect a new controller. When the platform confirms that a container stopped, it records a final receipt for that exact container and schedules recovery. If the runtime inspection is uncertain, the checker records `InternalError`; it does not assume that a team caused downtime.

3. EVIDENCE AND SCORING METHOD

3.1 Evidence counts and personal denominators

The score compares what a team achieved with the opportunities it actually had. In a fraction, the bottom number is the **denominator**. A personal denominator removes field-wide void rounds and any cooldown round forced on that team. For team i, hill h, and epoch e, define:

A team is **responsible** for a scorable tick when the checker observes its eligible code. If no new eligible code appears, the confirmed king remains responsible. A platform-void sample adds no responsible tick to the score.

Table 2. Fixed-formula evidence counts.

Symbol	Count
x_{ihe}	Number of acquisition windows in which the team confirmed a capture.
y_{ihe}	Number of acquisition windows in which the team had at least one scorable, non-cooldown chance to capture.
u_{ihe}	Number of scorable ticks controlled by the team's exact current capability.
s_{ihe}	Number of ticks the team could personally score after removing field-wide voids and its own cooldown exclusions.
b_{ihe}	Number of responsible ticks in which the checker returned `Ok`.
d_{ihe}	Number of scorable ticks for which the team was responsible.
w_h	Snapshotted service weight, constrained to $[0.8, 1.2]$.
z_{he}	Hill evidence switch: one when the hill has at least one scorable tick in the epoch, otherwise zero.

`InternalError`, reset and readiness time, incomplete token issuance, and platform-attributed failures count for no team. A champion cooldown is removed only from that champion's s_{ihe}. RSCTF removes an acquisition window from a team's denominator only when that team had no eligible scoring opportunity during the entire window.

3.2 Acquisition, control, and reliability

The acquisition, control, and reliability rates measure capture success, duration of control, and service health during responsibility:

$$A_{ihe} = \frac{x_{ihe}}{y_{ihe}}, \qquad C_{ihe} = \frac{u_{ihe}}{s_{ihe}}, \qquad R_{ihe} = \frac{b_{ihe}}{d_{ihe}}. \qquad (1)$$

Here, A is the acquisition rate, C is the control rate, and R is the reliability rate. An empty denominator produces zero. The implementation rejects counts that are negative or not finite, then limits each rate to $[0, 1]$. A team with no responsible tick has $R = 0$ and receives no points.

RSCTF first combines acquisition and control into the following intermediate value, B:

$$B_{ihe} = 0.25A_{ihe} + 0.55C_{ihe} + 0.20\sqrt{A_{ihe}C_{ihe}}. \qquad (2)$$

It then multiplies B by reliability to calculate the local hill score:

$$L_{ihe} = 100R_{ihe}B_{ihe}. \qquad (3)$$

Equation (2) makes sustained control more valuable than capture speed. A team with $A = 1, C = 0, R = 1$ scores 25; it captured every available window but held no scorable tick. A team with $A = 0, C = 1, R = 1$ scores 55; it controlled every scorable tick but earned no acquisition. The square-root term rewards doing both. Equation (3) then applies reliability to the entire result.

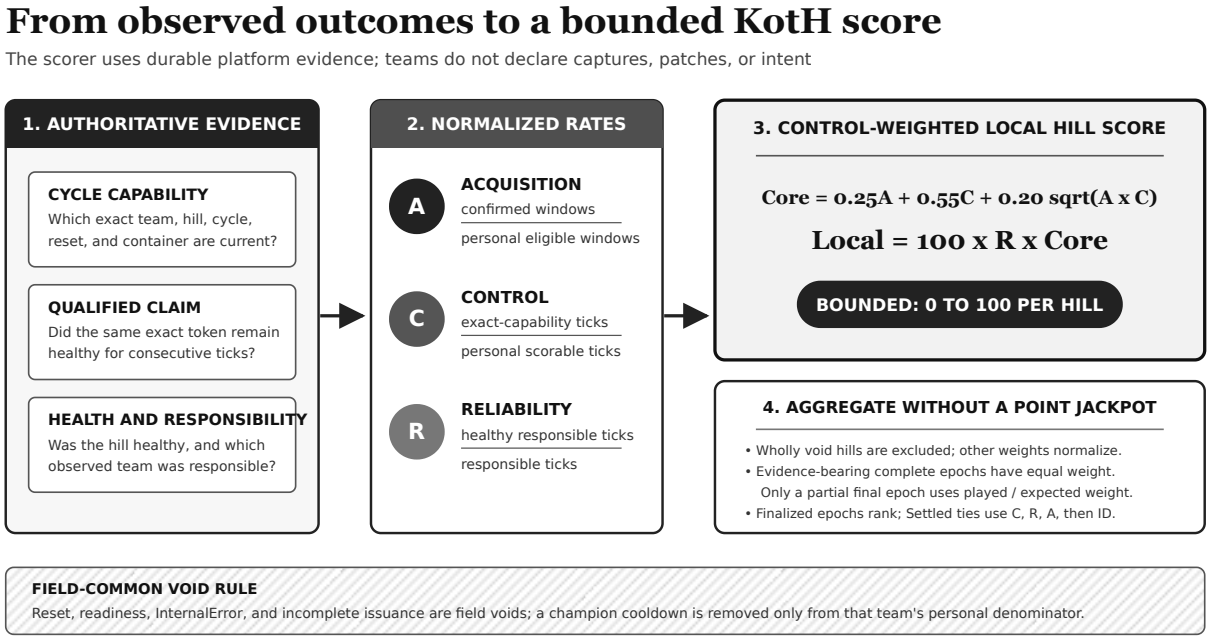

Figure 3. Fixed-formula pipeline. Exact cycle/container evidence becomes bounded personal rates before hill normalization and finalized-epoch aggregation.

3.3 Hill normalization

When an event has several hills, RSCTF combines their local scores without raising the 100-point epoch limit. Let H_e be the set of hills represented in epoch e. The normalized team score is:

$$E_{ie} = \frac{\sum_{h \in H_e} z_{he} w_h L_{ihe}}{\sum_{h \in H_e} z_{he} w_h}. \qquad (4)$$

If the denominator is zero, the epoch contributes zero. Service weights change how much each hill influences the result, but the epoch still cannot exceed 100 points. A hill with no scorable evidence for any team has $z_{he} = 0$, so it does not lower every team's score as an assumed failure. After the hill records one scorable tick, $z_{he} = 1$ and its approved weight applies. Equation (1) has already removed individual void samples from each team's denominator.

3.4 Epoch weight and totals

Every complete epoch with at least one scorable tick for the field has weight one. Other void samples can reduce its denominators but not its epoch weight. A complete epoch made entirely of field-wide voids has weight zero because it contains no performance that RSCTF can assign to a team. A final event may stop partway through an epoch. For that partial final epoch, let p_{he} be the number of played scorable ticks for hill h and let n be the configured epoch length. Its weight is:

$$q_e = \frac{\sum_{h \in H_e} w_h p_{he}}{n \sum_{h \in H_e} w_h}. \qquad (5)$$

For the set of included epochs J, the total is:

$$T_i(J) = \frac{\sum_{e \in J} q_e E_{ie}}{\sum_{e \in J} q_e}. \qquad (6)$$

RSCTF does not make late rounds worth more. For example, a three-tick final tail in the default twelve-tick epoch has weight $3 / 12 = 0.25$.

3.5 Settled score, projection, and rank

Settled includes only finalized epochs. It is the value used for official rank. **Projected** also includes unfinished evidence, so it can change while the current epoch continues. A projected value never breaks an official tie.

Equal Settled scores are resolved by higher control rate, higher reliability, more confirmed acquisition windows, and finally lower participation ID. RSCTF uses this evidence order for both the stable list order and the ordinal rank shown to players. Projected points do not enter the tie-break.

3.6 Reading the deployed scoreboard

The upper-right header shows only the `Epoch N` and `Tick x/y` badges. The deployed player board also shows the confirmed king, provisional claimant, cycle number and tick, reset or readiness phase, cooldown participants, next-reset countdown, Settled and Projected scores, and the corresponding A / C / R rates. Orange values identify projections; finalized ranking points use a separate visual style.

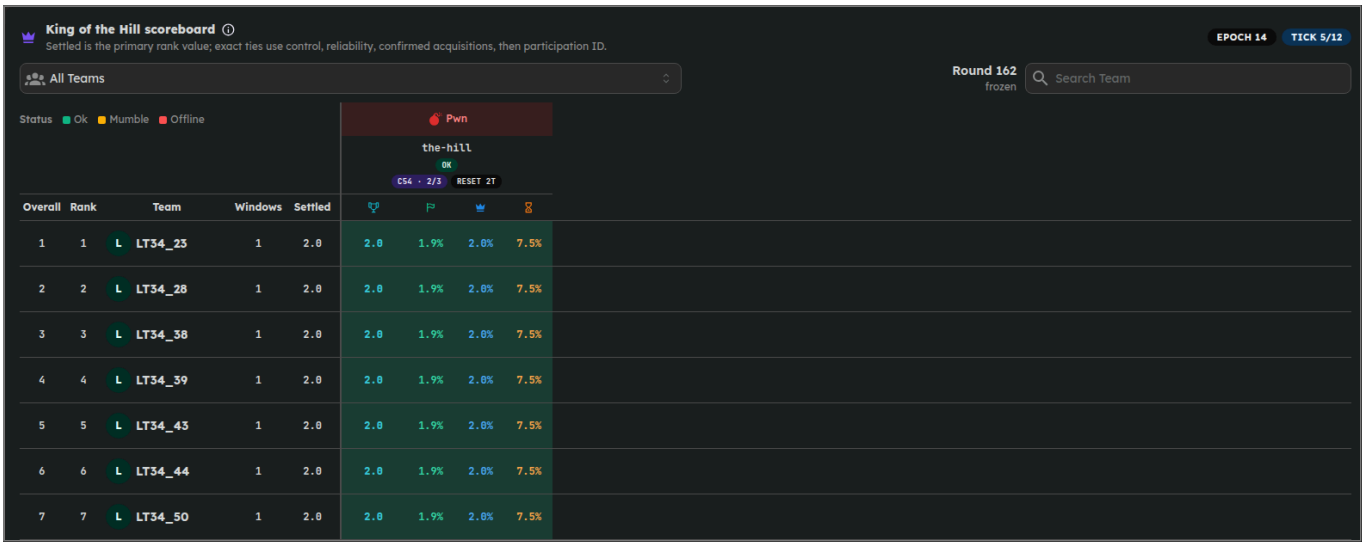

Figure 4. RSCTF fixed-formula scoring interface captured from the Docker Compose deployment on 14 July 2026 after the retained one-hour, 100-team event settled. The board shows ordinal ranks and the upper-right Epoch 14 and Tick 5/12 badges; the former cycle-length header badge is absent. The screen is application output, not a static mockup.

4. WORKED EXAMPLES

4.1 Provisional to confirmed

Cedar reaches confirmation across three checker samples under the default requirement of two consecutive healthy ticks.

Table 3. Cedar's claim from first observation through confirmed hold.

Sample	Board state	Evidence added
Tick 1: Cedar's current token and `Ok`	Provisional `1/2`	Cedar earns one controlled tick and one healthy responsible tick, but no acquisition yet.
Tick 2: same token and `Ok`	Confirmed king	Cedar earns another controlled and healthy responsible tick, plus one acquisition.
Tick 3: same token and `Ok`	Confirmed hold	Cedar's control and reliability counts grow. This window cannot award the acquisition again.

The verdict changes the sequence. A `Mumble` verdict at tick 2 would break the streak and return its progress to zero. An `InternalError` at tick 2 would leave the streak at one and exclude that sample from scoring.

4.2 One team on one hill

Suppose Cedar plays one hill for one epoch. Cedar confirms one capture during two eligible windows, controls three of eight eligible scorable ticks, and keeps the service healthy for four of five responsible ticks:

$$A = \frac{1}{2} = 0.5000, \qquad C = \frac{3}{8} = 0.3750, \qquad R = \frac{4}{5} = 0.8000\,. \tag{7}$$

$$\begin{aligned} B &= 0.25(0.5000) + 0.55(0.3750) + 0.20\sqrt{0.5000 \times 0.3750} \\ &= 0.1250 + 0.20625 + 0.086603 = 0.417853, \end{aligned} \tag{8}$$

$$L = 100(0.8000)(0.417853) = 33.4282\,.$$

The calculation gives Cedar a 50% acquisition rate, 37.5% control rate, and 80% reliability. Together, these rates produce **33.43 points out of 100** for this hill and epoch.

4.3 Personal cooldown denominator

Consider a three-tick cycle. The previous champion is blocked during tick 1, then controls the hill with an `Ok` service during ticks 2 and 3. RSCTF counts only the two ticks that the team could play, so its personal control denominator is two rather than three: $C = 2 / 2 = 1$. The acquisition window still counts because the team had scoring opportunities during ticks 2 and 3. The forced cooldown tick does not become a failed claim.

4.4 Two weighted hills

Now suppose the event has two hills. Hill Alpha has weight 1.2 and uses Cedar's local score from Equation (8). Hill Beta has weight 0.8 and a local score of 70:

$$E = \frac{(1.2)(33.4282) + (0.8)(70.0000)}{1.2 + 0.8} = 48.0569\,. \tag{9}$$

Alpha influences the result more than Beta because its weight is larger. The bounded weights do not raise the epoch's 100-point ceiling.

4.5 Equal complete epochs and a short tail

Suppose a team scores 80 and 60 in two complete epochs. The event then ends after three ticks of the next twelve-tick epoch, where the team has a projected score of 40. The two complete epochs each have weight 1, and the final tail has weight 0.25:

$$T = \frac{(1)(80) + (1)(60) + (0.25)(40)}{1 + 1 + 0.25} = 66.6667\,. \tag{10}$$

The short final tail contributes one quarter as much as either complete epoch.

5. FAILURE ADJUDICATION AND RECOVERY

5.1 Functional verdicts

Table 4. Checker verdict treatment.

Verdict	Meaning	Fixed-formula treatment
`Ok`	The service passed every behavior required by the checker.	Advances a matching provisional streak and records a healthy responsible tick.
`Mumble`	The service answered, but its response failed a required checker condition.	Breaks confirmation and records an unhealthy responsible tick when RSCTF can attribute it to a team.
`Offline`	The checker could not reach the service, or the platform confirmed that its managed backend stopped.	Breaks confirmation and records an unhealthy responsible tick when RSCTF can attribute it to a team.
`InternalError`	The platform or checker infrastructure could not produce evidence that RSCTF can safely assign.	Creates a void sample, pauses confirmation, and stays out of all denominators.

5.2 Durable reset state machine

RSCTF models a reset as a **state machine**: a fixed sequence of named phases stored in the database. The phases are finalization, snapshot, destroy, create, publish, capability issuance, readiness, firewall, active play, cooldown release, completion, failure, and event end. Each transition uses a durable **compare-and-set**, which changes the phase only when the stored phase still matches the expected value. RSCTF also writes an audit receipt identified by the cycle, phase, and reset attempt.

A local **single-flight guard** lets only one copy of the same task run inside an RSCTF process. A PostgreSQL **advisory lock**, scoped to one game and hill, lets only one RSCTF replica own a checker or reset transition. Together, these guards prevent duplicate work within one process and across several replicas. The code never holds a blocking lock while it waits for an asynchronous operation.

5.3 Identity fences

An **identity fence** ties evidence to the exact object and scoring period that produced it. Each checker record contains the container ID, game, challenge, round, cycle, reset attempt, and capability window. The checker and reset path also use the same hill lock. These rules prevent:

- checker/reset overlap for one hill;
- duplicate cycle or acquisition rows;
- stale tokens after reset;

- results from the old container entering the replacement's score;
- reset-time failure from entering a participant denominator; and
- late evidence after the event deadline.

5.4 Safe retry

The admin console shows the stored reset phase, old and replacement container IDs, readiness failures, champions, cooldown participants, provisional progress, and audit receipts. The **Retry** action calls the same recovery path used after a crash. This path is **idempotent**: repeating it has the same final effect as running it once. Retry does not use a separate repair procedure.

6. FAIRNESS AND INCENTIVES

6.1 Why control receives 55%

A one-time claim proves that a team reached the marker, but it does not prove sustained ownership. Control duration counts repeated observations of the exact token across available ticks. Its 55% direct coefficient makes persistence worth more than the 25% acquisition coefficient. The remaining 20% square-root term rewards teams that both take and retain the hill.

6.2 Why containers reset

Without resets, the first team to harden the service successfully could gain an advantage for the rest of the event. A fixed clean reset makes defense a skill that teams must repeat in each short cycle. Teams can still patch, test, and automate, but their changes last only until the next boundary. Every replacement uses the same configured challenge image, so a reset cannot substitute an easier or harder variant.

6.3 Why champions receive a cooldown

The reset removes the champion's accumulated filesystem changes. The default one-tick block then gives challengers the first opportunity to attack that hill, but it awards them no points. RSCTF also removes the blocked tick from the champion's personal denominator. The champion therefore loses both access and the corresponding scoring opportunity, rather than receiving an unavoidable failure.

6.4 Why the platform does not detect bots

Automation is permitted. Timing and behavior cannot reliably distinguish a script from a fast human or an assisted team. RSCTF therefore checks exact capabilities, rate limits, allowed network scope, and evidence integrity. It does not attempt to classify participants as humans or bots.

6.5 What the score establishes

Table 5. Interpretation limits.

Observed claim	What RSCTF establishes	What remains unknown
Confirmed acquisition	The checker observed one exact capability and a healthy service for the required consecutive rounds.	The exploit path, operator, novelty, or method used to bypass a patch.
Controlled tick	The same exact capability appeared before and after the functional service test.	Whether the team held continuous control between checker samples.
Responsible tick	RSCTF could assign control during that sample to the team.	Which person or tool changed the service.
Healthy responsibility	The organizer's checker returned `Ok`.	Whether the checker tests every property that organizers intend to protect.
Platform void	RSCTF could not safely assign the evidence.	What would have happened if the platform fault had not occurred.

7. PLAYER OPERATIONS

7.1 Reliable control loop

Use this loop for each hill:

1. Check the hill's token and state endpoints regularly.
2. Wait until the board shows that the cycle is active and your team is eligible.
3. Write the exact current capability to `/koth/king` promptly. Do not try to predict the checker's sample time.
4. Keep the service working throughout the full confirmation streak.
5. Monitor the holder, your team's responsibility, and the reset countdown.
6. After a reset, delete the old token from your tools and request the new token for that exact hill.
7. Apply only patches that you have tested and can reverse. Every filesystem change disappears at the next crown boundary.

A token for one hill does not work on another. Automated tools should store each credential under its game, challenge, cycle, and reset attempt so they never reuse a stale or incorrectly scoped token.

7.2 Treat generated actions as untrusted changes

AI assistance can speed up reconnaissance and patch drafting, but generated commands can stop the checked service, remove your access, or reveal the claim code. Test each generated change against the configured image. Review destructive commands, keep a rollback method, and confirm the result on the scoreboard. RSCTF scores the observed outcome, not whether a human or tool produced it.

7.3 Capability hygiene

A KotH token is a **bearer secret**: anyone who has the text can present the claim. Keep tokens out of public logs, screenshots, writeups, and shared continuous-integration output. A token stops working after reset, but RSCTF keeps its historical issuance record as audit evidence. Do not plant another team's token or attack infrastructure outside the published hill scope.

8. ORGANIZER AND REVIEW PROCEDURE

8.1 Before scoring

Before play, publish the tick duration, epoch length, cycle length, cooldown, confirmation threshold, service weights, checker contract, allowed network scope, and appeal process. Then verify that:

- every ranked KotH challenge has exactly one managed active container;
- the replacement image matches the configured image;
- the cycle divides the epoch and every range validation passes;
- at least two accepted teams receive distinct per-hill capabilities;
- the VPN/firewall can enforce a hill-specific champion cooldown;
- the marker can be read before and after a functional probe; and
- readiness succeeds on a pristine replacement.

The official snapshot stores these settings as the rules for that game. Changes made later in the editor cannot rewrite finalized evidence.

8.2 During play

Monitor the stored reset phase, current and replacement container IDs, readiness failures, confirmation progress, cooldown release, result completeness, and event deadline. Read the audit receipts before using the admin retry action. Retry resumes the same transition; it does not start a new reset. Do not create a replacement manually while the durable reset process owns the hill.

8.3 Finalization

At the event deadline, RSCTF closes unfinished reset and confirmation work and rejects late evidence. A partial final epoch keeps only the fraction that teams actually played. A hill or epoch with no scorable evidence for the field carries no weight. Publish awards only from finalized epochs, after the board reports full settlement.

Keep the following records through the appeal period: official configuration; team-roster and hill snapshots; cycle rows; cooldown sets; token issuance and revocation; control observations; acquisition receipts; container filesystem diffs; reset receipts; and finalized score rollups.

9. VALIDATION AND LIMITATIONS

The fixed formula has bounded scores and exact identity checks. Those properties establish implementation integrity, but they do not show how a particular challenge will behave in competition. During test events, organizers should measure confirmation failures, control changes, time to the first challenger after reset, repeat winners, cooldown use, void samples, readiness latency, reset failures, and score sensitivity to cycle length.

One randomized checker sample per tick cannot prove continuous ownership between samples. The functional checker defines "healthy" through the behaviors it tests, so it may omit a relevant service property. Champion cooldown creates an opening only when the network layer enforces the block. The default three-tick cycle and two-tick confirmation threshold leave little time to exploit and repair the service. Organizers should test each challenge to determine whether its tick duration provides enough time for both tasks.

10. CONCLUSION

RSCTF's fixed formula keeps a shared KotH challenge contestable through fixed crown-cycle resets. Every reset replaces the old container with one clean container built from the same configured image. Under the default settings, the previous champion sits out one tick that is also removed from its personal denominator. A new claim must pass two consecutive healthy observations before RSCTF confirms the acquisition. Every scoring observation identifies its exact container, cycle, reset, round, and token.

The score combines 25% acquisition, 55% control, and a 20% balance term. It then multiplies that full result by the service reliability achieved while the team was responsible. Complete epochs containing evidence have equal weight. Wholly void hills do not count, and short final epochs receive proportional weight. Unfinished results remain projections. Equal finalized scores use control, reliability, acquisition, and participation ID as ordinal tie-breaks. These rules reward teams for taking the hill, holding it, and keeping it working. They do not rely on bot detection or permanent patches.

APPENDIX A. FREQUENTLY ASKED QUESTIONS

A.1 Is a tick the same as a round?

Yes. Both words mean one opportunity for the scheduler to run the checker. A crown cycle contains several ticks, and an epoch contains one or more crown cycles.

A.2 Does one token work on every hill?

No. RSCTF issues each capability for one team, one hill, one cycle, and one reset attempt.

A.3 Does a marker write immediately award acquisition?

No. The first scorable observation of an eligible current token creates a provisional claim. Only the configured streak of healthy observations confirms acquisition.

A.4 What breaks confirmation?

A different valid token, an ineligible claimant, `Mumble`, or `Offline` breaks the streak. `InternalError` and incomplete token issuance create void evidence, so they pause rather than break it.

A.5 What survives a crown reset?

Audit records and scoring evidence survive. The old container, marker, patches, and active capabilities do not.

A.6 Can the previous champion play during cooldown?

Yes, on the rest of the game. The champion cannot access or claim the cooled hill until the authoritative cooldown round ends. RSCTF removes that blocked tick from the team's personal denominator.

A.7 How are tied champions handled?

Every team tied for the lead enters the champion set. If blocking the full set would leave no challenger, RSCTF disables cooldown for that cycle.

A.8 Does destroying the container erase responsibility?

No. Before recovery, RSCTF records evidence tied to the exact container that it confirmed as stopped. If the runtime state is uncertain, RSCTF creates a platform void instead of assuming that the responsible team caused downtime.

A.9 Is there a late-epoch multiplier?

No. Complete epochs containing scorable evidence have equal weight. RSCTF excludes an epoch that is entirely void for the field. Only a partial final epoch receives a fractional weight.

A.10 Can automation participate?

Yes. RSCTF does not attempt unreliable bot detection. Automated tools remain subject to the same credentials, allowed network scope, and rate limits as manual play.

APPENDIX B. NOMENCLATURE

Table 6. Symbols and board terms.

Symbol or term	Definition
i, h, e	Labels for one team, hill, and epoch.
A	Share of the team's eligible acquisition windows that produced a confirmed capture.
C	Share of the team's eligible scorable ticks that it controlled.
R	Share of the team's responsible ticks in which the service was healthy.
B	Combined acquisition-and-control value before applying reliability.
L	Score for one team, hill, and epoch, bounded to 0-100.
w_h	Snapshotted service weight in $[0.8, 1.2]$.
z_{he}	One when hill h has scorable evidence for the field in epoch e; zero otherwise.
E_{ie}	The team's normalized score for one epoch.
q_e	Weight assigned to a complete or partial epoch.
Provisional claimant	Team whose exact eligible token was observed but has not completed the healthy confirmation threshold.
Confirmed king	Team whose exact claim completed the required healthy observations.
Settled	Official score calculated only from finalized epochs.
Projected	Informational score that also includes unfinished evidence.

APPENDIX C. IMPLEMENTATION TRACEABILITY

This appendix connects each rule in the handbook to the code that implements it. It is intended for developers, reviewers, and event operators, so it retains repository and API terminology. Paths are relative to the repository revision that contains this

handbook.

Table 7. Fixed-formula source-of-truth map.

Responsibility	Source of truth
Configuration validation and official snapshot	`src/services/ad/engine/koth_cycle/config.rs`
Cycle coordinates and tied-champion selection	`src/services/ad/engine/koth_cycle/state.rs`
Durable reset lifecycle and audit receipts	`src/services/ad/engine/koth_cycle/lifecycle/`
Qualified claim transition and acquisition idempotency	`src/services/ad/engine/koth_cycle/claims.rs`
Champion cooldown release	`src/services/ad/engine/koth_cycle/cooldown.rs`
Exact-container checker and immutable observations	`src/services/ad/engine/checker/koth.rs`
Pure A, C, R, local, hill, and epoch formulas	`src/controllers/game/koth/scoring_formula.rs`
SQL evidence and personal denominators	`src/controllers/game/koth/scoring/evidence.rs`
Finalized epoch rollups	`src/controllers/game/koth/scoring/rollup/`
Ordinal tie-break construction and board cells	`src/controllers/game/koth/board.rs`
Player and admin DTOs/routes	`src/controllers/game/koth/mod.rs`, `admin.rs`
Crown schema and reset-attempt integrity	`src/migrations/m0046_koth_crown_cycles.rs` through `m0058_constant_koth_scoring.rs`

C.1 Core HTTP surface

Table 8. KotH routes used by players and operators.

Method and route	Purpose
`GET /api/Game/{id}/Ad/Koth/Token`	Returns the caller's active exact-hill capability rows.
`GET /api/game/{id}/ad/koth/{challengeId}/token`	Returns one hill's `{round, token, status}` capability state.
`GET /api/game/{id}/ad/koth/{challengeId}/state`	Returns confirmed/provisional holder state, cycle progress, eligibility, cooldown, reset phase, and countdown.
`GET /api/Game/{id}/Ad/Koth/Hills`	Lists enabled hills and current lifecycle state.
`GET /api/game/{id}/ad/koth/scoreboard`	Returns fixed-formula cadence, settlement, lifecycle, ranks, and A / C / R detail.
`GET /api/game/{id}/ad/koth/timeline`	Returns cumulative finalized/projected score history.
`GET /api/edit/games/{id}/ad/koth/state`	Returns the operator lifecycle and scoring view.
`POST /api/edit/games/{id}/ad/koth/{challengeId}/recover`	Calls the idempotent recovery path.

The API uses camelCase field names, string-valued enumerations, and timestamps represented as Unix milliseconds.

C.2 Verification scope

The implementation test suite checks malformed formula inputs, formula bounds, confirmation, interrupted streaks, one acquisition per token, personal cooldown denominators, single and tied champions, partial epochs, ordinal tie-breaks, and void evidence. PostgreSQL tests check exact reset windows, event-deadline closeout, container identity, and foreign-key integrity.

The JavaScript lifecycle harness runs capture, confirmation, cycle reset, stale-token rejection, concurrent polling, BYOC tunnels, health checks, and duplicate/integrity queries against the Docker Compose deployment.

REFERENCES

1. RSCTF Project, "Crown-cycle King of the Hill implementation," repository-local source artifact, fixed scoring formula, verified 13 July 2026.
2. K. Bock, G. Hughey, and D. Levin, "King of the Hill: A Novel Cybersecurity Competition for Teaching Penetration Testing," in *Proceedings of the 2018 USENIX Workshop on Advances in Security Education*, Baltimore, MD, USA, 2018. [Online]. Available: https://www.usenix.org/conference/ase18/presentation/bock. Accessed: 13 July 2026.
3. CTFd, "King of the Hill," *CTFd Documentation*, n.d. [Online]. Available: https://docs.ctfd.io/docs/custom-challenges/king-of-the-hill/. Accessed: 13 July 2026.
4. FAU Security Team, "Rules," *FAUST CTF 2025*, 2025. [Online]. Available: https://2025.faustctf.net/information/rules/. Accessed: 13 July 2026.